

CPU-First Reinforcement Learning for Native PC-98 Bullet-Hell Games

A Reproducible Systems Study and Research Plan

Touhou PC-98 RL Project

Working draft — 22 August 2026

Abstract

We study reinforcement learning (RL) directly against a native PC-98 bullet-hell game under a CPU-only constraint. The principal systems problem is not merely policy optimization: a useful experiment must boot a legally supplied disk image, identify the correct emulated-memory region, provide isolated resets, extract observations at game rate, and avoid CPU oversubscription. We present an end-to-end TH05 environment that uses exact child-process attachment, compact entity-set observations, a 180,136-parameter recurrent actor-critic, and multi-process proximal policy optimization (PPO). Compact reads have a measured median latency of 0.516 ms, compared with 3.550 ms for the dense-map path, while reducing transferred floats by a factor of 776. Eight emulators deliver about 223 transitions/s on the tested host. After 16,384 training transitions and validation-based checkpoint selection, an untouched four-seed, 900-step paired test improves mean scalar return from 0.0529 to 0.3639 and halves observed deaths. A completion-gated Stage 1 Lunatic curriculum subsequently produces short-phase no-miss clears. Moving the start from phase 3 to phase 2 and fine-tuning for 32,768 transitions improves held-out completion from six to eight of eight and reduces deaths from 19 to 14, but yields no no-miss clear. No full-stage run clears. We document negative results, validity threats, and a path from the current TH05 adapter toward a common TH01–05 framework.

1 Research questions

This project is organized around six falsifiable research questions.

- RQ1: Systems feasibility.** Can an unmodified emulator plus structured memory extraction sustain useful RL throughput on CPU without interfering with other workloads on a shared host?
- RQ2: Representation efficiency.** Can a compact, permutation-invariant representation of nearby hazards replace large, sparse spatial maps while preserving enough state for bullet-hell control?
- RQ3: Learning signal.** Under a small CPU budget, does recurrent PPO improve survival and resource management over a paired untrained policy?
- RQ4: Safe exploration.** Do adapter-certified hard action constraints improve sample efficiency without merely hiding an unsafe policy?
- RQ5: Sparse survival credit.** Can training-only auxiliary targets spread a rare miss signal backward in time without sacrificing task completion or injecting a scripted dodge policy?
- RQ6: Generality.** Which interfaces can be shared by Touhou 1–5, and which memory, termination, reward, and patching components must remain game-specific?

The present evidence addresses RQ1 strongly, RQ2 at the systems level, and RQ3 only preliminarily. RQ4 has an underpowered comparative result. RQ5 is motivated by the observed completion/no-miss gap but not yet tested. RQ6 is deliberately deferred until the agent can complete TH05 on its highest difficulty; only TH05 is currently validated.

2 Background and related work

The initial reverse-engineering reference describes a high-fidelity, multi-objective bullet-hell environment and an accelerator-oriented baseline [3]. We treat that work as a source for low-level TH05 state discovery, not as the architecture or identity of the present framework.

Our optimizer follows clipped PPO [6] with generalized advantage estimation (GAE) [5]. A game frame contains a variable-size collection of bullets and special projectiles. Processing these objects as sets follows the invariance motivation of Deep Sets [7]; learned attention pooling is related to Set Transformer [2], although our encoder is deliberately smaller and does not perform quadratic self-attention among all hazards.

DOSBox-X supplies PC-98 emulation [1]. ReC98 release P0335 is used only as a clean boot baseline [4]; it is not a runtime or build dependency.

3 Failure analysis of the reference baseline

The reference project solved valuable reverse-engineering problems, but its released implementation encodes assumptions that do not hold for our CPU-only, multi-process, current-emulator setting. We audited those assumptions and reproduced their consequences before replacing them. Table 1 separates observed failure modes from design preferences; “inadequate” here means inadequate for the stated experimental setting, not universally wrong.

Table 1: Reference-baseline assumptions, observed consequences, and our replacements.

Baseline assumption	Why it fails here	Replacement
XPU/CUDA-only package and dense CNN input	No supported GPU is available; each 2,048-step map rollout was documented as 1.62 GiB	CPU PyTorch lockfile, compact entity sets, and a small GRU
At most 8–10 MiB per scanned memory region	The current DOSBox-X guest mapping is about 30 MiB, so game signatures are never searched	Writable anonymous-region scan with a tested 64 MiB safety cap
Shared host game directory	Concurrent emulators can read and write the same configuration and score files	One private HDI copy and exact child PID per environment
Dense $24 \times 92 \times 96$ maps every step	Allocation and language-boundary transfer dominate CPU inference	273-float native compact reader
Normalize every objective before weighting	Reward magnitude loses its semantic meaning and sparse/noisy objectives receive unit variance	Vector GAE, fixed semantic scalarization, then one normalization
Overwrite a latest-only checkpoint	A later update can silently replace a better earlier policy	Periodic snapshots, validation LCB, untouched test
No deadline for a live-but-stalled worker	The learner can wait forever while leaving emulator children behind	Bounded startup/step IPC and exact cleanup
Readable memory implies rollout readiness	About 80 opening action slots ignore control but still emit survival reward	Reset gate waits for the player input lock to end
Patch stage values are human numbered	The loader writes them directly to TH05’s zero-based resident stage, silently shifting scenarios by one	Human Stage 1–6 mapped to resident 0–5; both values recorded

3.1 Version-fragile systems assumptions

The released memory walker discards mappings larger than 10 MiB during its initial search and uses an 8 MiB limit for subsequent readable-region filters. On the tested DOSBox-X release, the relevant writable guest mapping is roughly 30 MiB. This is a hard compatibility failure rather than a throughput issue: the correct signature cannot be found in bytes that are never searched. The new 64 MiB cap is still bounded, but its value is checked against the actual mapping layout. Searches are further restricted to writable anonymous regions, retaining a small candidate set.

The original spawn path correctly attaches to its newly created child, but all workers mount one host `export` directory. Its separate attach-existing path chooses the first process returned by a name search. The former risks cross-worker file-state interference and the latter is ambiguous whenever unrelated

emulators run on the host. Private writable disk images plus exact child ownership make environment identity explicit. This matters for science as well as reliability: a reset is not reproducible if another worker can alter its persistent state.

The same audit found unbounded coordination waits for workers that remain alive but stop responding, and setup instructions that rely on privileged execution and world-writable build directories. Our runner instead uses child-process tracing, finite deadlines, deterministic cleanup, and ordinary user permissions. These changes reduce both stalled experiments and accidental interference with unrelated jobs.

Memory readability is also weaker than actionability. In a live directional probe, the initial player state was readable immediately but remained fixed for about 80 action intervals while the opening invincibility/input lock was active. The baseline collector records this prefix and attributes its survival reward to sampled actions that the game cannot execute. Our reset gate waits for both an initialized player and the end of this lock. A sustained-left/right probe then moved normalized X from 0.50 to 0 and back beyond 0.62, independently confirming that the subsequent action path is effective.

3.2 CPU cost is determined by representation

The reference package pins an XPU build of PyTorch and states that a GPU is required. More importantly, its learning path stores a $24 \times 92 \times 96$ tensor for every transition, even though most cells are empty. Merely selecting a CPU wheel would therefore leave the principal allocation, copy, memory-bandwidth, and CNN costs intact. Our change is algorithmic as well as infrastructural: nearby hazards are variable-size sets, so a set encoder expresses the symmetry directly and makes CPU inference smaller. Table 2 reports the resulting measured reader cost.

3.3 Optimization semantics and model selection

For objective o , the reference trainer first constructs $\hat{A}^{(o)} = \text{standardize}(A^{(o)})$, combines these standardized values, and standardizes the sum again. Consequently, multiplying an objective’s raw reward by almost any positive constant has no effect, and a rare or noisy objective can be promoted to the same empirical variance as a dense, reliable one. This is not intrinsically invalid, but it no longer implements scalarization in declared reward units. We retain a vector critic and vector GAE, apply explicit semantic weights to the unnormalized advantages, and normalize the resulting policy advantage exactly once.

Finally, overwriting a single “latest” file assumes that training progress is monotone. Our short runs directly contradict that assumption: a policy that improved at 24,576 transitions regressed badly by 49,152. Periodic immutable snapshots, a disjoint validation split, and an untouched paired test do not eliminate selection bias, but they make it visible and prevent the last update from becoming the result by accident. Snapshot ranking is lexicographic: completion count, no-miss completion count, then lower-confidence-bound return. Workers retain episode-local death state so training metrics distinguish a successful clear from a no-miss clear even when an episode spans rollouts.

4 System design

4.1 Reproducible native-game execution

The environment starts from a user-provided Japanese HDI. A fail-closed patch script verifies complete SHA-256 hashes before applying the required executable layouts. The supplied archive is a mixed revision: `MAIN.EXE` requires patch layout 1, whereas `OP.EXE` matches layout 2. Treating layout numbers as a paired version produces a black screen. Patched binaries and game data are never committed.

Each Gymnasium environment copies the 21 MiB template HDI into a private temporary directory. This makes resets independent and prevents concurrent workers from racing on game-written score or configuration files. DOSBox-X is spawned as a direct child, and the native extension attaches to that exact PID. The memory search is restricted to writable anonymous mappings no larger than 64 MiB. This includes the approximately 30 MiB guest-memory mapping used by the tested DOSBox-X release; the previous 8–10 MiB cap silently excluded it.

Startup and step operations have explicit timeouts. Reset retries cover both process creation and the interval before initialized gameplay becomes readable. Workers release inputs and terminate their exact child process on shutdown.

4.2 Compact observation

The compact observation has 273 floats:

$$37 \text{ global features} + 16 \times 7 \text{ special-projectile features} + 16 \times 7 \text{ bullet features} + 12 \text{ item features}. \quad (1)$$

Entities are ordered by distance for truncation, but the neural encoder pools them as a set. Each entity contains relative position, velocity, type, speed, and normalized distance. Missing entities use the sentinel $(0, 0, 0, 0, 0, 1)$. A previously plausible mask based only on nonzero values is wrong because the sentinel distance is one; the implementation now recognizes the full sentinel and has a regression test for all-padding inputs.

The older observation additionally builds 24 maps of size 92×96 , or 211,968 floats per step. Our hot path constructs no map. This is important because map allocation and Python transfer can cost more than policy inference on CPU.

4.3 Policy and value model

Special projectiles and bullets pass through separate shared token networks. Each set is summarized by learned attention pooling and masked max pooling. Global state and item features have small multilayer perceptrons (MLPs); the four summaries are fused into a 128-dimensional vector. A one-layer GRU with 128 hidden units models temporal context. The actor predicts 19 discrete actions. A shared critic head predicts three value components rather than collapsing objectives before value estimation. The model contains 180,136 parameters.

4.4 CPU recurrent PPO

For scaled vector reward $\mathbf{r}_t \in \mathbb{R}^3$, vector GAE is

$$\delta_t = \mathbf{r}_t + \gamma(1 - d_t)\mathbf{V}(s_{t+1}) - \mathbf{V}(s_t), \quad (2)$$

$$\mathbf{A}_t = \delta_t + \gamma\lambda(1 - d_t)\mathbf{A}_{t+1}. \quad (3)$$

The policy advantage is $A_t = \mathbf{w}^\top \mathbf{A}_t$, normalized once after scalarization. This avoids independently normalizing sparse objectives and then assigning them equal empirical variance. PPO uses ratio clipping, value clipping, entropy regularization, gradient clipping, and approximate-KL early stopping.

Rollouts are synchronous across independent emulator processes. Every worker uses one PyTorch thread; the learner defaults to eight. The parent process is CPU-affined and run with positive niceness. Periodic snapshots are selected on a validation seed set using

$$S = \bar{R} - z \frac{s_R}{\sqrt{n}}, \quad (4)$$

after prioritizing observed stage successes. Thus the last PPO update is not automatically considered the best model.

Two post-baseline extensions are exposed for controlled ablation. Analytic geometry appends bounded constant-velocity closest-approach features using physical scales supplied by the game adapter. Hard safety masks only actions the adapter can certify as unavailable or behaviorally redundant—currently a bomb with zero stock and movement into an active boundary clamp. The masked categorical is renormalized identically during rollout and PPO optimization, and removed probability mass is logged. It does not implement scripted collision avoidance.

5 Experimental protocol

Host. The test machine exposes 96 logical CPU threads and approximately 755 GiB RAM. Other experiments occupied selected high-numbered cores. RL jobs were run at niceness 10 and confined to known-idle low-numbered cores. No GPU was visible to PyTorch. The software stack used Python 3.14.6, PyTorch 2.13.0+cpu, and DOSBox-X tag `dosbox-x-v2026.07.02`.

Game condition. The first short-run dataset actually starts at Stage 2 (resident stage index 1) with three lives, three bombs, character 2, rank 0, and power 0. It was initially labelled Stage 1 by following the patch README; source audit showed that the loader assigns `skip.to` directly to TH05’s zero-based resident field. We retain and relabel those measurements rather than discard or rewrite them. New scenario tooling accepts human Stage 1–6 and writes resident 0–5. The environment advances at a 36 ms action interval. Evaluations are stochastic and paired by PyTorch sampling seed. The game itself is not yet randomized through a controlled seed API.

Training. The selected policy comes from training seed 202 at update 8: 16,384 total transitions from eight workers. Six snapshots were compared for 900 steps on validation seeds 41, 43, and 44. The final comparison uses untouched seeds 51–54. These test seeds were not used for training or model selection.

Metrics. We report observation latency, transferred floats, model parameters, rollout throughput, scalar return, death count, and the raw resource-reward component. Stage success is the primary task metric but is absent from all short runs. The exact aggregate and per-seed values are tracked in `experiments/2026-08-22-th05-cpu.json`.

6 Results

6.1 Systems efficiency

Table 2: Paused-game observation-reader benchmark.

Reader	Median (ms)	95th percentile (ms)	Floats returned
Dense maps	3.550	5.014	211,968 + 273
Compact	0.516	0.542	273

Compact reading is 6.88 times faster by median and transfers 776 times fewer map-equivalent floats. The compact actor-critic has 180,136 parameters versus 2,375,478 for the comparison CNN-GRU model, a 13.2-fold reduction.

Eight live environments sustain approximately 223 transitions/s when no reset occurs. A 2,048-transition rollout takes about 9.19 s, while three PPO epochs take 0.20–0.33 s. Therefore environment time, not gradient computation, is currently dominant. Increasing learner threads from one to eight improved the model microbenchmark, but 16 threads caused severe oversubscription; bounded threading is part of the algorithmic system, not a cosmetic setting.

6.2 Snapshot selection

Table 3: Validation results for the six candidate snapshots (three seeds, 900 steps each). LCB uses $z = 1$.

Candidate	Mean return	Standard error	LCB score
Reproduction update 8	0.2333	0.2341	-0.0007
Reproduction update 10	0.2112	0.2573	-0.0461
Reproduction update 12	0.0167	0.0145	0.0022
Seed 101 update 8	0.2408	0.2379	0.0030
Seed 202 update 8	0.2743	0.2362	0.0381
Seed 303 update 8	0.2769	0.2401	0.0368

Selection first maximizes clears, then no-miss clears, then lower-confidence return. In this experiment all completion counts tie at zero, so seed 202 is selected because it has a slightly higher lower-confidence score than seed 303 despite a lower mean. This illustrates why latest-only or single-seed checkpointing is unreliable at this budget.

Table 4: Untouched stochastic test. Resource values closer to zero are better.

Seed	Untrained			Selected PPO		
	Return	Deaths	Resource	Return	Deaths	Resource
51	0.0085	1	-157	0.0135	1	-157
52	0.0165	1	-155	0.6875	0	-87
53	0.1985	1	-83	0.7325	0	-69
54	-0.0120	1	-168	0.0220	1	-152
Mean/total	0.0529	4	-140.75	0.3639	2	-116.25

6.3 Untouched paired test

The paired mean return gain is 0.3110 with standard error 0.1707. Deaths fall by 50%, and mean resource cost improves by 17.4%. With only four seeds, the uncertainty is large; a normal 95% interval for return gain would cross zero. The correct conclusion is preliminary improvement, not statistical certainty.

6.4 Negative and unstable results

A hand-written short-horizon collision heuristic performs worse than random in the tested condition, reaching four deaths and terminal failure around 948–1,107 steps. Independent nearest-bullet prediction does not capture multi-projectile escape geometry; the heuristic is retained only as a negative ablation and is not used for behavior cloning.

An earlier exploratory policy improved four-seed return after 24,576 transitions, then regressed badly by 49,152 transitions. Three additional 16,384-transition seeds varied widely. These observations motivated periodic snapshots, validation-based selection, and an untouched test split.

6.5 Matched Lunatic ablation and curriculum probe

Plain compact recurrent PPO and the analytic-geometry plus hard-safety variant were each trained with seeds 1011, 2022, and 3033 for 24,576 transitions. We retained updates 4, 8, and 12, selected nine candidates per condition on seeds 71–73, and tested the selected pair on untouched seeds 81–84 for 1,200 steps. Source audit relabelled this condition as Stage 2 (resident index 1).

Table 5: Matched TH05 Lunatic Stage 2 ablation. LCB is validation mean minus one standard error; deaths are totals across four test seeds.

Condition	Validation mean	LCB	Test mean	Test deaths
Plain PPO	-0.856	-1.449	-1.648	14
Geometry + hard safety	-0.822	-1.422	-1.054	11

Neither condition clears the stage. The paired test gain is 0.594 with standard error 0.669, and one seed contributes most of it. Thus the point estimate favors geometry plus safety but does not establish a reliable gameplay improvement. The shield constrains 40.1% of test decisions while removing only 2.47% of policy probability on average, consistent with a narrow legality constraint rather than a scripted dodge policy.

A true Stage 1 Lunatic probe at phase 3, ending at phase 4, is cleared by a hard-safety random policy in 397 steps with one death. This confirms that phase termination provides a dense success signal. It motivates a completion-gated curriculum: learn no-miss completion on a short phase, move the starting phase backward, reduce privileged power/resources, and only then optimize a full stage. The machine-readable record is `experiments/2026-08-22-th05-lunatic-ablation.json`.

We then trained two geometry-plus-safety PPO pilots on that phase. Ten retained snapshots were ranked on seeds 111–114. The selected update clears all four runs without a miss in three. Table 6 reports two disjoint eight-seed tests.

Against the untrained policy, selected PPO gains 0.290 mean return with paired standard error 0.100 and quadruples no-miss clears. This is evidence of learned safety on the short phase, not merely access to an already easy completion. However, the lower-entropy, higher-learning-rate pilot does not replicate its four-seed validation advantage: on seeds 131–138 it trails default PPO by 0.093 return (paired standard

Table 6: Stage 1 Lunatic phase 3–4 results. Each row contains eight stochastic episodes; deaths are totals.

Comparison	Clears	No-miss	Deaths	Mean return
Untrained matched policy	8	1	7	1.661
Selected PPO	8	4	4	1.952
Default PPO update 8	8	3	6	1.790
Phase-tuned PPO update 8	8	1	7	1.697

error 0.136) and two no-miss clears. We therefore reject any claim that this hyperparameter change is reliably better.

Frequent terminals lower mean collection throughput to 76–80 transitions/s, versus 223 without resets; one rollout with twelve successes takes 64.4 s. This measured 2.8–2.9-fold slowdown strengthens H3: phase curricula need an asynchronous, sequence-aware collector rather than additional learner threads. Exact results are in `experiments/2026-08-22-th05-lunatic-curriculum.json`.

We next froze that checkpoint and evaluated transfer to the longer phase 2–4 task. It clears seven of eight episodes versus six for a matched untrained policy, with 16 versus 19 deaths, but neither condition clears without a miss. The paired return gain is 0.566 with standard error 0.639, so this transfer comparison is inconclusive.

For a clean curriculum transition, weights were loaded while Adam moments, learning rate, update counters, and recurrent state were reset. A 32,768-step default-PPO run produced four snapshots. Validation seeds 301–304 select update 4 by the predeclared clear/no-miss/return-LCB ordering. Although update 16 has higher mean return (2.514 versus 2.373), its higher uncertainty gives a lower one-standard-error LCB (2.333 versus 2.347).

Table 7: Untouched Stage 1 Lunatic phase 2–4 fine-tuning test, eight stochastic episodes per row.

Policy	Clears	No-miss	Deaths	Mean return
Frozen phase 3–4 initializer	6	0	19	1.739
Phase 2–4 fine-tuned PPO	8	0	14	2.609

Table 7 uses untouched seeds 311–318. Fine-tuning gains 0.870 paired return with standard error 0.464. This is evidence for improved completion, not no-miss mastery: every episode still contains at least one death. Curriculum therefore made completion feedback dense but did not solve survival credit assignment. Exact records are in `experiments/2026-08-22-th05-lunatic-p2-curriculum.json` and its linked raw selection and paired reports.

7 Threats to validity

- The full-stage paired test has only four stochastic seeds; each short-phase test has eight. Standard errors remain material, and no full-stage clear occurs.
- Evaluation seeds control action sampling, but the native game does not yet expose a separately controlled environment RNG seed.
- The comparison model has no reproducible compatible checkpoint, so systems comparisons do not imply a gameplay win over prior work.
- Learning evidence is limited to TH05 Stage 2 full-stage training and privileged Stage 1 phase 2–4 tasks. It does not establish transfer to the start of the stage, lower resources, other characters or ranks, or TH01–04.
- The compact observation drops most hazards beyond the nearest 16 in each category and currently omits a compact normal-enemy set.
- Host timing is measured while unrelated jobs are present. CPU affinity and niceness reduce interference but do not produce a dedicated-machine benchmark.

8 Research roadmap

The next experiments are specified as hypotheses rather than assumed upgrades.

H1: richer compact geometry. Evaluate the implemented closest-approach features, then add normal-enemy tokens while keeping the observation below 2 KiB. Compare each addition against the 273-float schema with matched training transitions and validation/test splits.

H2: TH05 Lunatic mastery. Train a stage/phase curriculum that ends at rank 3 with no privileged starting resources. Promote a policy only after stage completion, then optimize no-miss completion and finally full-game completion. Use completion and resource-normalized completion as primary metrics rather than short-prefix return.

H3: reset-aware collection. When terminals become common, replace global synchronous waiting with an asynchronous collector or reset pool. The prediction is improved wall-clock throughput without changing the on-policy age bound.

H4: sparse-miss auxiliary credit. For each on-policy transition, predict whether a miss occurs within 15 and 30 future actions. Mask samples whose horizon crosses an unfinished rollout boundary, stop gradients through labels, and expose no future information at inference. Compare PPO and PPO plus the shared-encoder auxiliary loss at matched transitions. The hypothesis is rejected if no-miss completion does not improve or ordinary completion regresses.

H5: risk-aware objectives. Compare the fixed scalarization with constrained or distributional survival objectives. Any method must report the full reward vector and may not hide a survival regression behind score gain.

H6: adapter-certified safety. Compare no mask, exact resource/boundary constraints, and any future predictive shield at matched training budgets. Report removed policy probability mass and evaluate every trained policy both with and without its shield.

H7: old-five-games interface. Define per-game contracts for observation schema, action meaning, terminal flags, executable identity, and reset. TH01–04 enter the benchmark only after live parser/action tests pass and TH05 mastery has been addressed; shared code alone is not evidence of support.

9 Conclusion

Native PC-98 RL is practical on a CPU when observation construction, process attachment, worker isolation, and thread scheduling are treated as first-class research concerns. The present system removes a large dense-map bottleneck, runs eight independent games at useful speed, and exhibits small-data improvements in short-phase completion and survival. The persistent gap between completion and no-miss completion motivates an auxiliary-credit ablation rather than a claim of mastery. The main scientific task is to turn this systems baseline into a pre-registered multi-configuration study with full-stage completion metrics.

References

- [1] DOSBox-X contributors. DOSBox-X: Pc-98 emulation guide, 2026. Accessed 2026-08-22.
- [2] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiosek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3744–3753, 2019.
- [3] T. Liu. A high-fidelity reinforcement learning environment and baseline for multi-objective bullet hell games, 2026.

- [4] ReC98 contributors. ReC98 release p0335, 2026.
- [5] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [7] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabás Póczos, Ruslan Salakhutdinov, and Alexander J. Smola. Deep sets. In *Advances in Neural Information Processing Systems*, volume 30, 2017.